



UNIVERSIDADE
FEDERAL RURAL
DE PERNAMBUCO

UFRPE-UAST

Esta é uma marca d'água para a versão de teste, compre para obter a versão completa.

Benefícios para usuários registrados:

1. Converte os documentos inteiros.

2. Nenhuma limitação para os documentos finais.

Celso Brennand

Remover Agora

Implementações dos Processos Distribuídos

AULA Nº 07

Os slides são baseados no livro Sistemas Distribuídos, de TANENBAUM, A. S. e STEEN, M. V.

Na aula anterior, vimos:

- **Threads e processos distribuídos**
- **Virtualização**

Conteúdo desta aula:

- **Clientes**
- **Servidores**
- **Migração de Código**
- **Sockets**

Comunicação entre Processos

“Coração” de qualquer Sistema Distribuído

- Como processos em diferentes máquinas trocam informações?
 - Devido à ausência de memória compartilhada, toda comunicação em SDs é baseada no envio e recebimento de mensagens (de baixo nível)

Comunicação entre Processos

Como processos em diferentes máquinas trocam informações?

- **Processo A** monta uma MSG em seu próprio espaço de endereçamento
- Executa uma chamada de sistema, SO envia a MSG pela rede até **B**
- **A** e **B** precisam concordar com significado de bits enviados
- **Vários acordo:**
 - Quantos volts são necessários para sinalizar um bit 0 e um bit 1?
 - Como o receptor sabe qual é o último bit da mensagem?
 - Como o receptor pode detectar se uma msg foi danificada ou perdida?
 - Qual é o comprimento dos números, cadeias e outros itens de dados, e como eles são representados?

Modelo OSI

- **International Organization for Standardization – ISO**
 - **Modelo de referência para interconexão de sistemas abertos (ISO OSI ou Modelo OSI)**
- **Modelo OSI**
 - **Cada camada lida com um aspecto específico**
 - **Problema quebrado em pedaços gerenciados independentemente**
 - **Cabeçalhos são adicionados em cada nível**
 - **Transmissão pela camada física**
 - **Cada camada retira e examina seu próprio cabeçalho**

Modelo OSI

- **Pilha de protocolos**
 - Modelo referência != protocolos utilizados
- **Protocolos desenvolvidos para internet (TCP/IP)** são os mais usados
- **3 camadas inferiores implementam funções básicas de rede**
 - Camada **Física**
 - Camada de **Enlace**
 - Camada de **Rede**



Modelo OSI

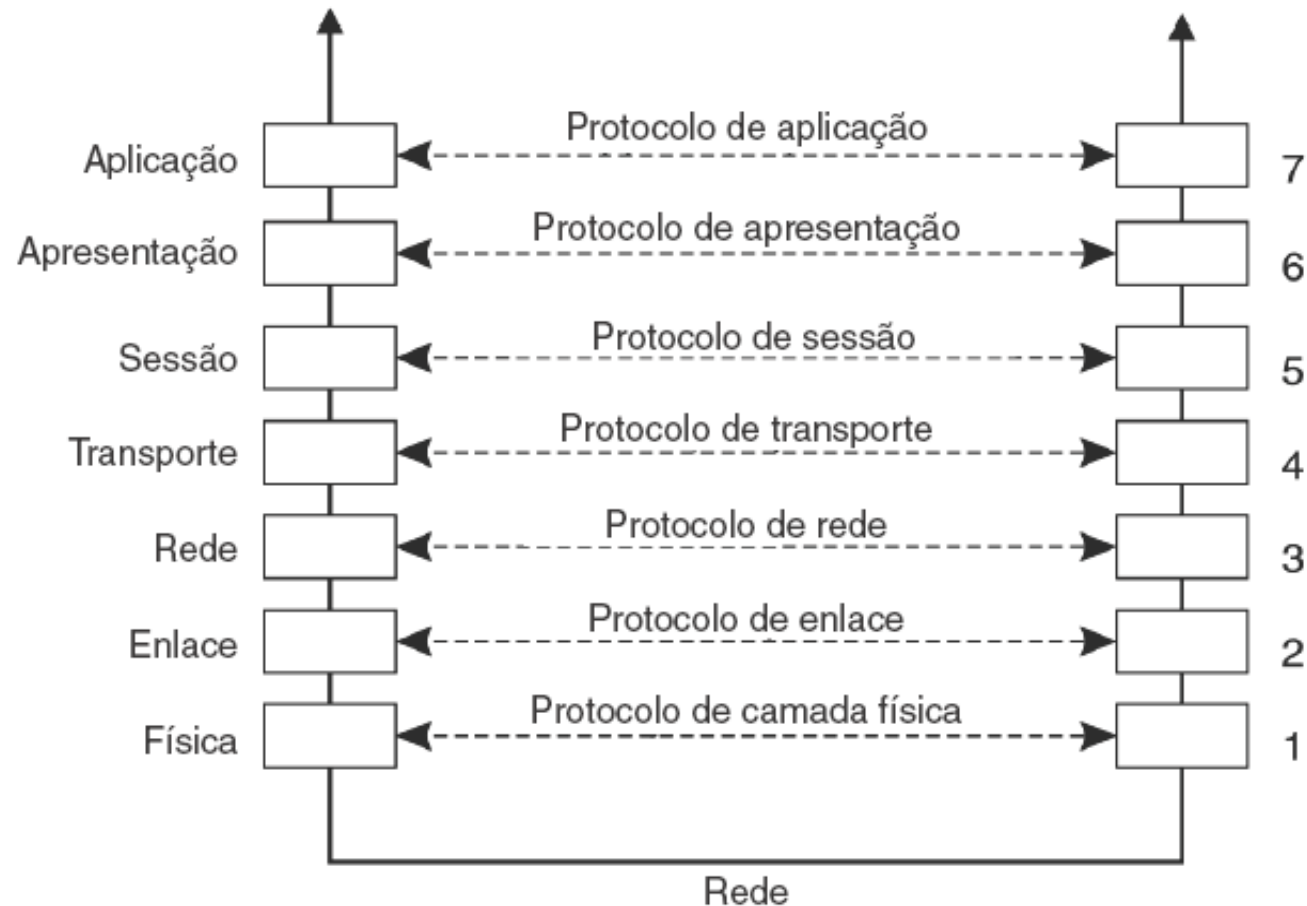


Figura 4.1 Camadas, interfaces e protocolos no modelo OSI.

Camada Física

- Responsável pelo envio de bits
- Questões fundamentais na camada física
 - Quantos volts usar para 0 e para 1
 - Quantos bits por segundo
 - Ambas as direções simultaneamente?
 - Tamanho/forma do conector, número de pinos...
 - Trata da padronização das interfaces elétrica, mecânica e de sinalização

Camada de Enlace

- Agrupar bits em unidades (quadros) e providenciar para que cada quadro seja corretamente recebido
- Coloca padrão especial de bits no início e no final de cada quadro e calcula uma soma de verificação
- Quando o quadro chega, o receptor soma novamente a soma de verificação dos dados e compara com a soma anexada
- Número no cabeçalho para identificar sequencia

Camada de Enlace

- Em uma LAN, em geral não há necessidade de o remetente localizar o receptor
 - Remetente coloca quadro na rede e o receptor o retira
- Em uma rede de longa distância?
 - Para ir do remetente até o receptor, uma mensagem talvez tenha de fazer alguns saltos escolhendo, em cada um, uma linha de saída para usar.
 - A questão de como escolher o melhor caminho é denominada **roteamento** e é, em essência a tarefa primária da camada de rede.

Camada de Rede

- Redes diferentes: mensagem pode fazer saltos
- Como definir um caminho entre um **par origem destino**?
- **Roteamento**: tarefa primária da camada de rede
 - Rota mais curta nem sempre a melhor
- Mais utilizado: IP – internet protocol
 - Roteamento independente, por pacote
 - Pacote – termo para uma mensagem na camada de rede

Camada de Transporte

- Responsável pela comunicação lógica entre diferentes processos sendo executados em diferentes hosts (fim-a-fim)
- Quebra mensagem, numera e envia
- Conexões de transporte confiáveis: rede orientada a conexão ou não – sequencia correta garantida ou não. Camada de transporte ordena.
- Transmission Control Protocol – **TCP** – conexão
- Universal Datagram Protocol – **UDP** – sem conexão

Camadas adicionais

Internet agrupou camadas superiores (apresentação, sessão, aplicação) em uma única camada

- **Sessão**: controle de diálogo – sincronização – pontos de verificação em transferências
- **Apresentação**: significado dos bits – definir campos da mensagem
- **Aplicação**: um conjunto de aplicações padronizadas
 - Engloba qualquer aplicação que não se ajuste a uma das camadas subjacentes
- Da perspectiva do modelo OSI, middlewares são aplicações

Camada de Middleware

- Camada de software que é situada logicamente entre uma camada de nível mais alto, composta de usuários e aplicações e uma camada subjacente, que consiste de facilidades básicas de comunicação
- Inúmeros protocolos para suportar serviços de middleware:
 - Autenticação: não estão vinculados a uma aplicação específica
 - Protocolo distribuído de bloqueio de recurso (**atomicidade**)
 - **Comunicação**: Serviços de comunicação de alto nível,
Evitar acessos simultâneos a um recurso

Camada de Middleware adaptado na pilha de protocolos

As camadas de apresentação e sessão foram substituídas.

Protocolos independentes da aplicação.

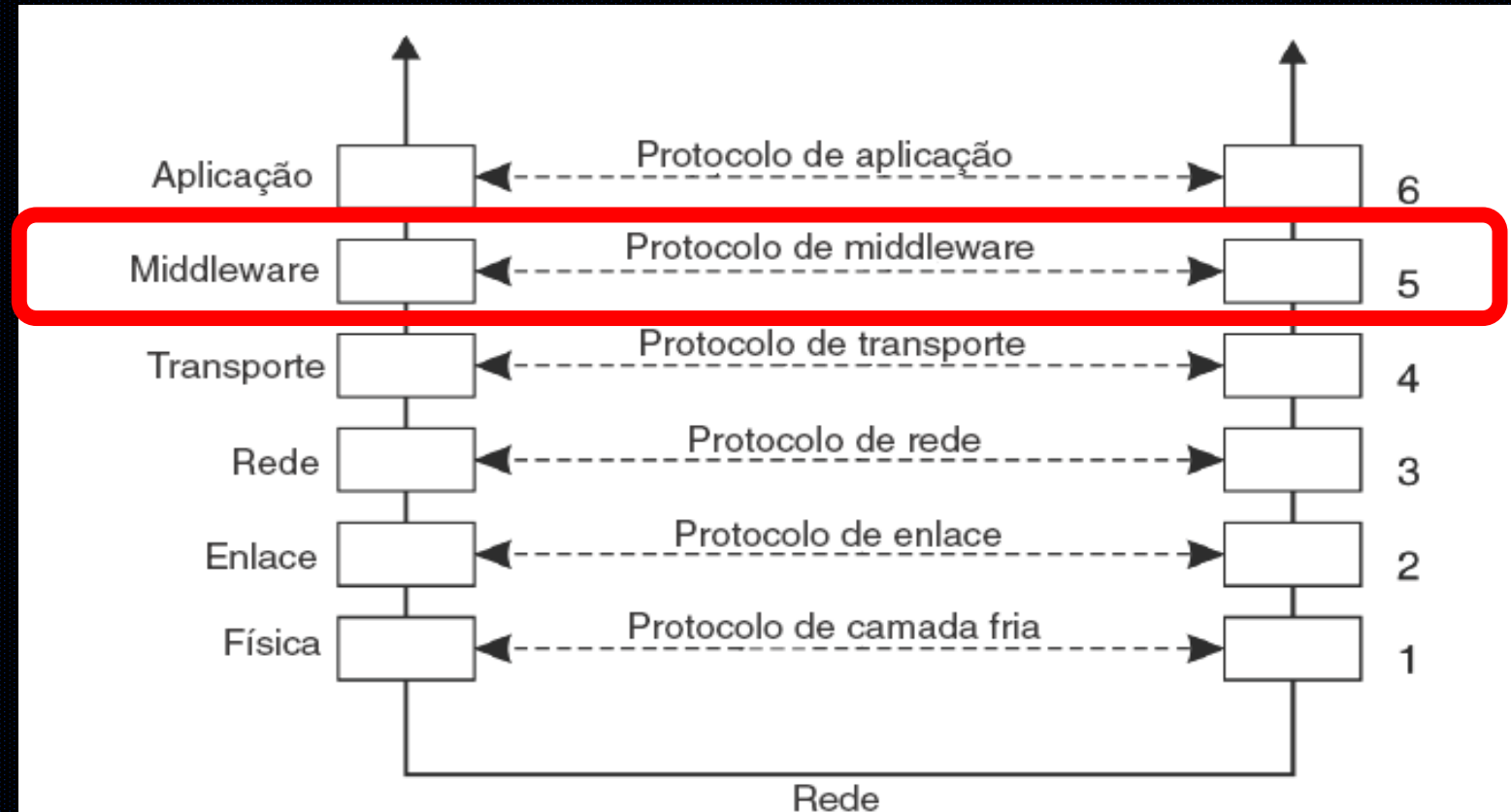
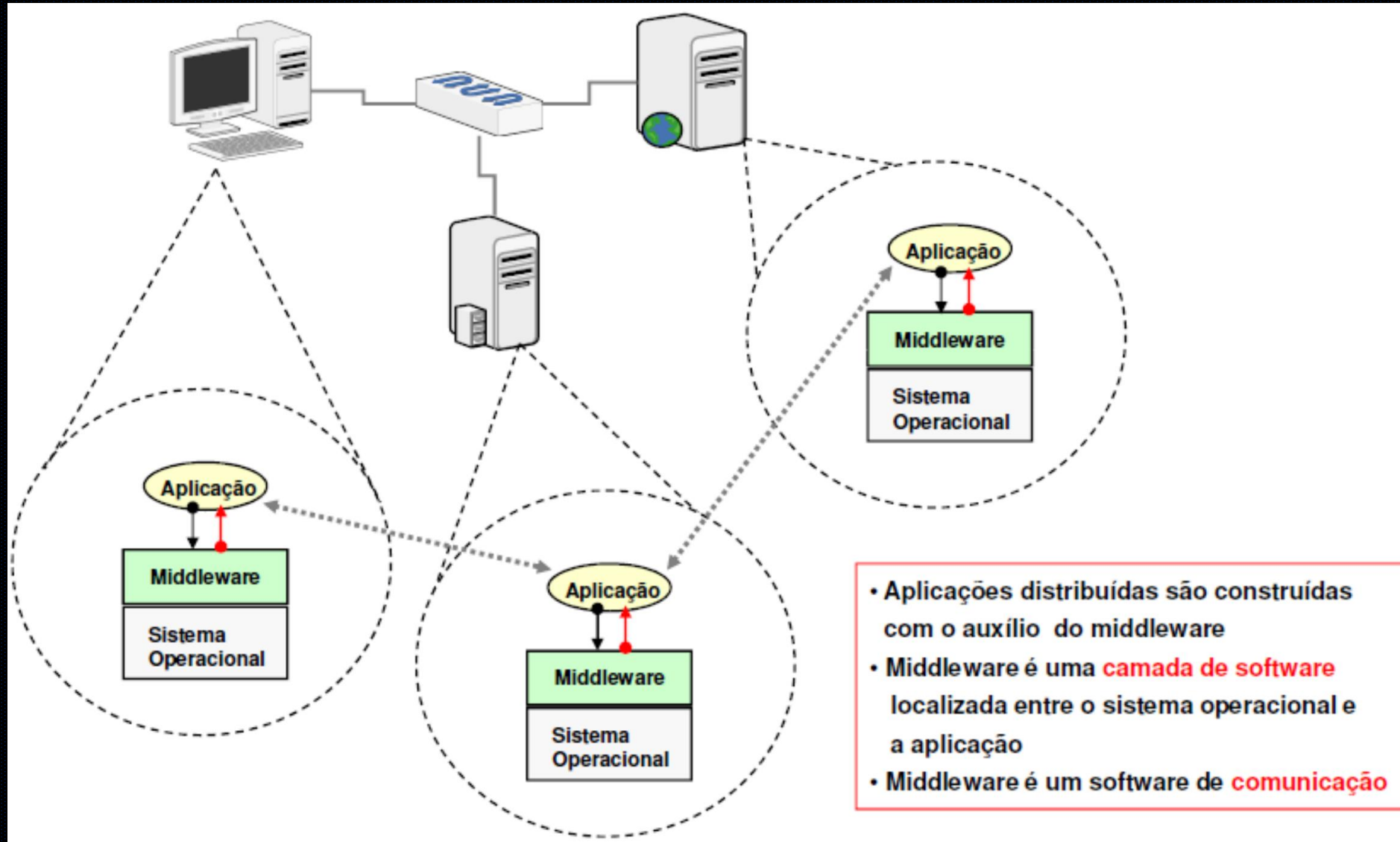


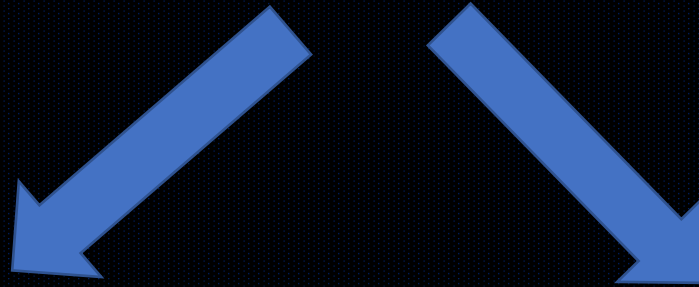
Figura 4.3 Modelo de referência adaptado para comunicação em rede.

Camada de Middleware



Alternativas de comunicação

Quais são as “**alternativas de comunicação**” entre processos executando na mesma máquina ou máquinas diferentes?



IPC “tradicionais”

- Pipe
- Socket
- memória compartilhada
- fila de mensagens
- etc

Middleware

- Chamada de Procedimento Remoto
- Comunicação orientada a Mensagem
- Comunicação Orientada a Fluxo

Middleware de Comunicação

Serviços de comunicação que podem ser adicionados a camada de Middleware e seus tipos:

- Chamadas de procedimento remoto
- Comunicação orientada a mensagens
- Comunicação orientada a fluxo

Tipos de Comunicação (Persistência)

Persistente: Mensagem armazenada durante o tempo que for necessário para entregá-la ao receptor.

Transiente: Mensagem armazenada somente durante a execução do remetente e do receptor.

Mensagens são armazenadas ou não pelo middleware.

Tipos de Comunicação (Sincronização)

Assíncrona: Remetente continua sua execução imediatamente após enviar mensagem.

Síncrona: Remetente é bloqueado até saber que sua requisição foi aceita.

Pontos de sincronização:

- Remetente bloqueado até que o middleware avise
- Remetente bloqueado até a requisição chegar no receptor
- Remetente bloqueado até o receptor responder.

Tipos de Comunicação (Granuralidade)

- **Discreta:** Comunicam por mensagens, onde cada uma delas é uma unidade de informação completa.
- **Fluxo:** Comunicam por várias mensagens que estão relacionadas uma com as outras.
 - Relacionadas pela ordem ou pela relação temporal.

Chamada de Procedimento Remoto (RPC)

É um middleware de comunicação que “**permite a processos chamar procedimentos localizados em outras máquinas**”, Birrell and Nelson (1984).

A ideia é fazer com que uma chamada de procedimento remoto pareça com uma chamada local (**transparência**).

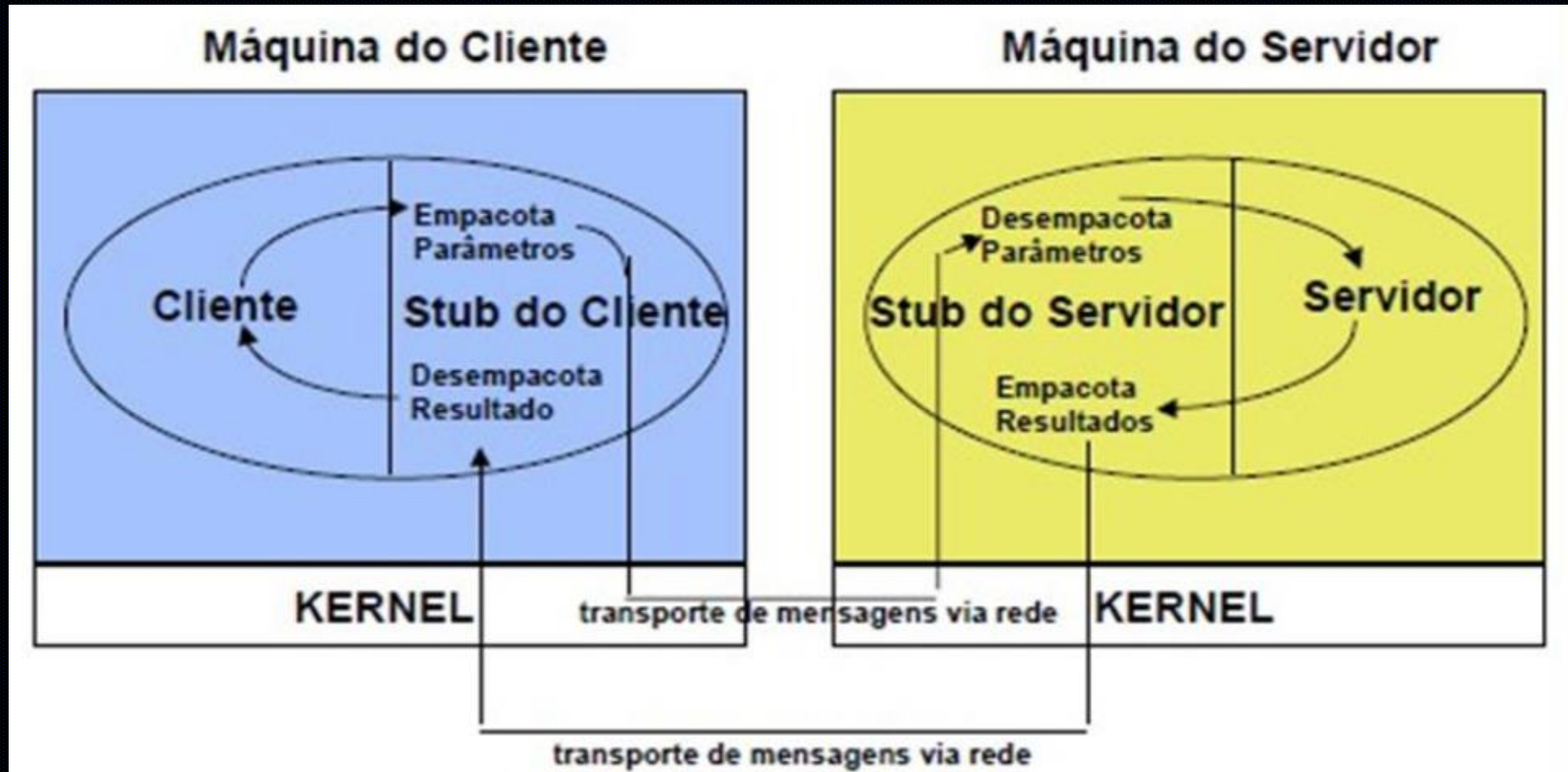
Problemas: Passar parâmetros em espaços de endereçamento diferentes, “queda da máquina”, etc.

RPC e os Stubs

A transparência é alcançada através dos stubs (apêndices):

- **Stub do cliente:** Empacota os parâmetros em uma mensagem e a envia para a máquina do servidor.
 - Desempacota e compatibiliza a arquitetura.
- **Stub do servidor:** Desempacota os parâmetros e invoca o procedimento correto passando os parâmetros *
- Empacota os parâmetros na mensagem de resposta
- Ambos são compilados antes.

Passos para uma RPC

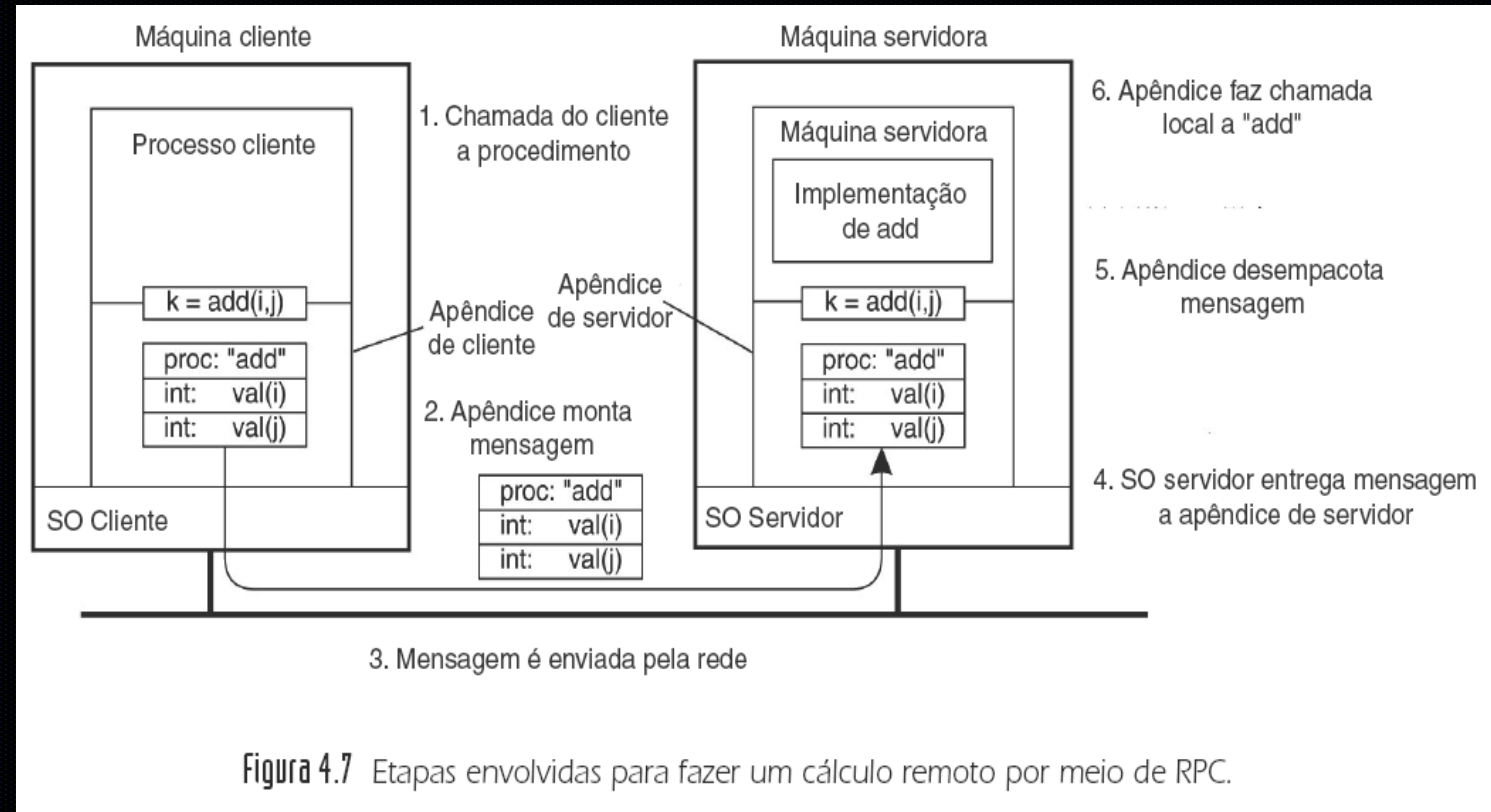


Passos para uma RPC

Empacotar parâmetros em uma mensagem é conhecido como **marshalling de parâmetro**.

RPC suporta:

- Passagem por valor
- Passagem por referência.



RPC – Linguagem de Programação de Interface - IDL

Interface: É um conjunto de procedimento que pode ser chamado por um cliente e implementado por um servidor.

IDL é a linguagem voltada para especificar a interface e permite definir procedimentos como **idempotentes**.

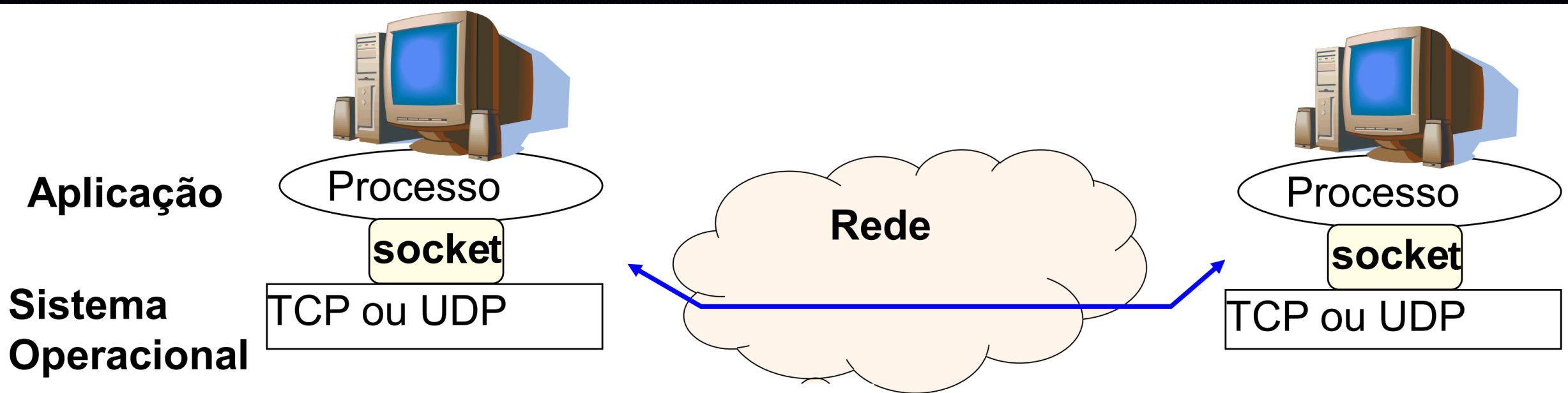
Há aplicações, como o stubgen, que geram o stub a partir da IDL (descrição própria que independe da linguagem).

Sockets

- **Socket** não é middleware, mas pode ser usado para construir aplicação distribuída (haja disposição!)
- Construir aplicação distribuída usando socket é equivalente a construir aplicações comuns usando **assembly**
- **Socket**: ponto final de uma comunicação full-duplex entre dois processos
 - Processo → casa / Socket → Porta
 - Socket: Porta entre o processo da aplicação e o protocolo de transporte

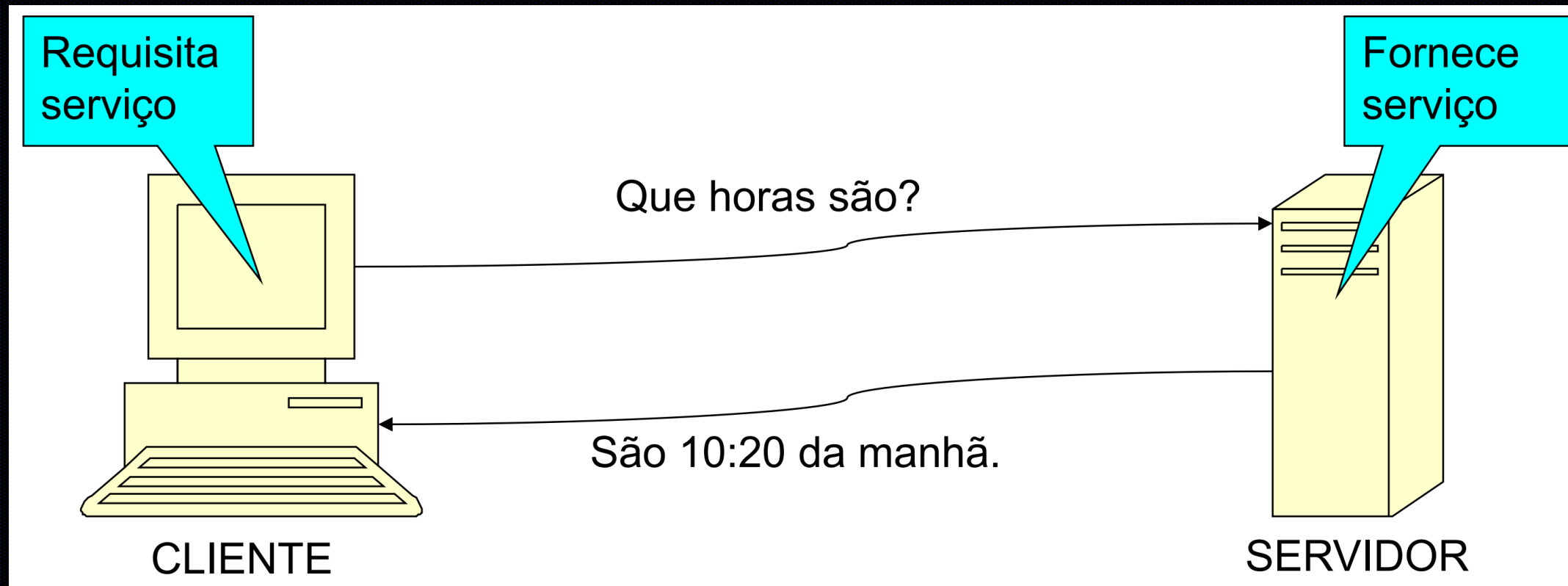
Sockets

- São abstrações que representam pontos de comunicação através dos quais processos se comunicam
- Para que dois computadores possam trocar informações
 - Cada um utiliza um socket



Sockets

- Os sockets adotam o paradigma cliente/servidor.



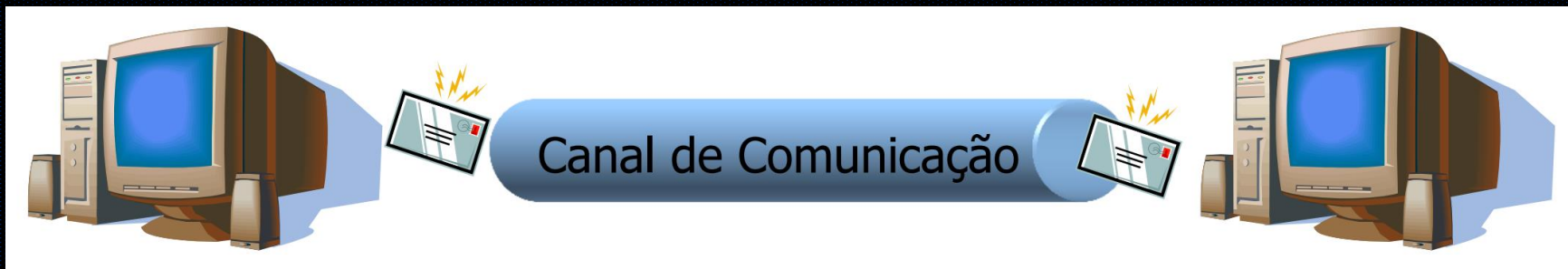
- Os papéis de clientes e servidores não são fixos em processos: um servidor pode ser cliente de outro serviço.

Sockets na Internet

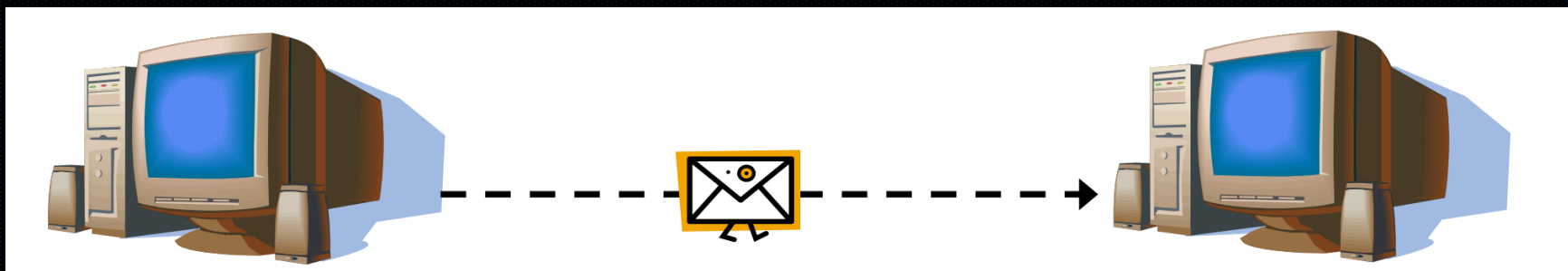
- Na Internet, cada máquina têm um endereço único:
 - **Endereço IP**
- Cada **endereço IP** possui portas associadas, que representam conexões individuais com este endereço
 - echo(7) ftp(21) telnet(23) http(80) pop3(110)
- Definidas pela IANA (Internet Assigned Numbers Authority)
- Para se comunicar com um servidor, um cliente precisa:
 - **Endereço IP** da máquina onde o servidor executa
 - **Número da porta** que o servidor escuta. Ex: 221.13.45.6:80
- **Binding**: quando o socket é criado, uma porta deve ser associada

Modos de Operação

- **Orientado a conexão:** TCP (Transport Control Protocol)
 - As mensagens são enviadas através de um canal de comunicação



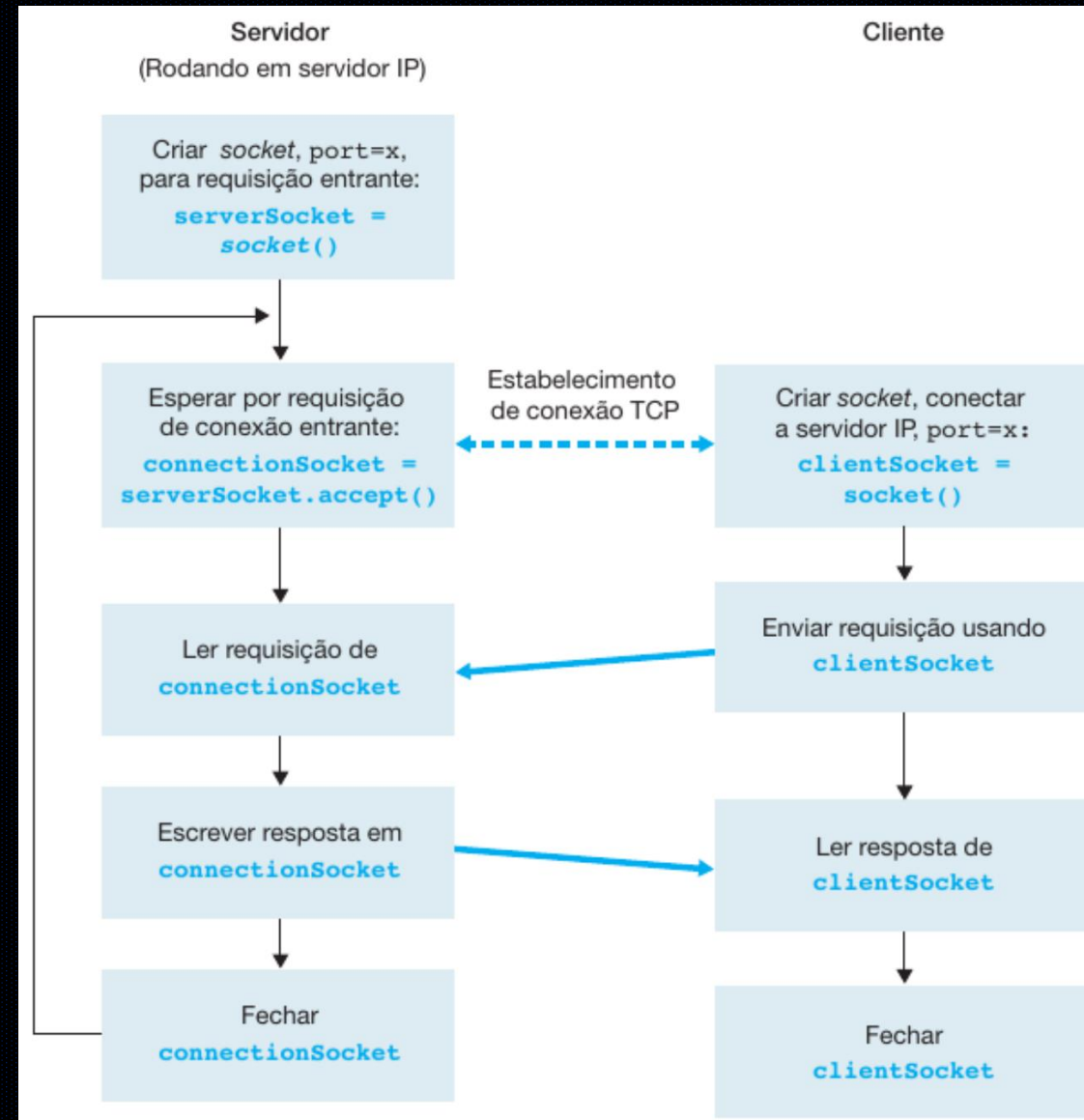
- **Sem conexão:** UDP (User Datagram Protocol)
 - As mensagens de tamanho fixo (datagramas) são transmitidas individualmente para destinos específicos



Orientado a conexão: TCP

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print ("O servidor está pronto para receber mensagens.")
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024)
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence)
    connectionSocket.close()
```

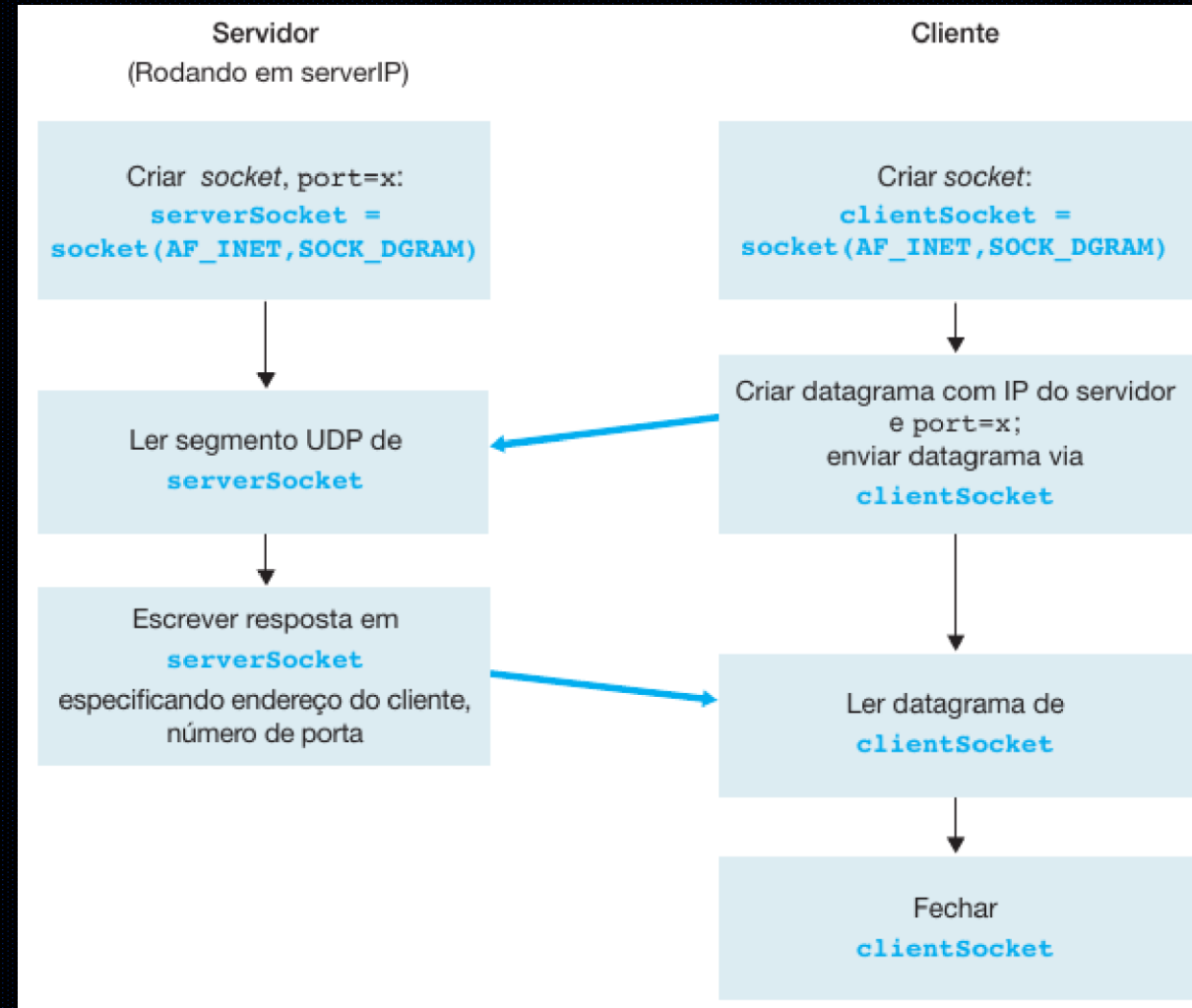
```
import socket
serverName = socket.gethostname()
serverPort = 12000
clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input("Entre com a mensagem usando letras minúsculas:")
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ("Resposta do Servidor:", modifiedSentence.decode())
clientSocket.close()
```



Sem conexão: UDP

```
import socket
HOST = ""
PORTA = 5000
meuservidorudp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
meuservidorudp.bind((HOST, PORTA))
while True:
    mensagem, clienteEndereco = meuservidorudp.recvfrom(128)
    mensagemModificada = mensagem.decode().upper()
    meuservidorudp.sendto(mensagemModificada.encode(), clienteEndereco)
    meuservidorudp.close()
```

```
import socket
HOST = socket.gethostname()
PORTA = 5000
meusocketudp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
mensagem = input("Escreva uma mensagem em minúsculo:")
meusocketudp.sendto(mensagem.encode(), (HOST, PORTA))
MensagemModificada, EnderecoServidor = meusocketudp.recvfrom(128)
print(MensagemModificada.decode())
meusocketudp.close()
```



Sockets na Internet

- Na Internet, cada máquina têm um endereço único:
 - **Endereço IP**
- Cada **endereço IP** possui portas associadas, que representam conexões individuais com este endereço
 - echo(7) ftp(21) telnet(23) http(80) pop3(110)
- Definidas pela IANA (Internet Assigned Numbers Authority)
- Para se comunicar com um servidor, um cliente precisa:
 - **Endereço IP** da máquina onde o servidor executa
 - **Número da porta** que o servidor escuta. Ex: 221.13.45.6:80
- **Binding**: quando o socket é criado, uma porta deve ser associada

Clientes: modo de interação com o servidor

Clientes: modo de interação com o servidor

Processos dos clientes → Permite que os meios interajam com servidores remotos a partir de dois modos:

- **Fat Client:** Para cada serviço remoto → um servidor separado com que possa estabelecer contato
 - Aplicação é mais robusta e possui seus próprios protocolos.
- **Thin Client:** Cliente provê apenas uma interface
 - O middleware auxilia na interação.

Clientes: modos de interação

com o servidor

Esta é uma marca d'água para a versão de teste, compre para obter a versão completa.

(a) Benefícios para usuários registrados: Protocolo de interação é implementado no nível da aplicação e é específico a uma aplicação.

1. Converte os documentos inteiros.

(b) Protocolo de interação é implementado no nível do middleware e é genérico.

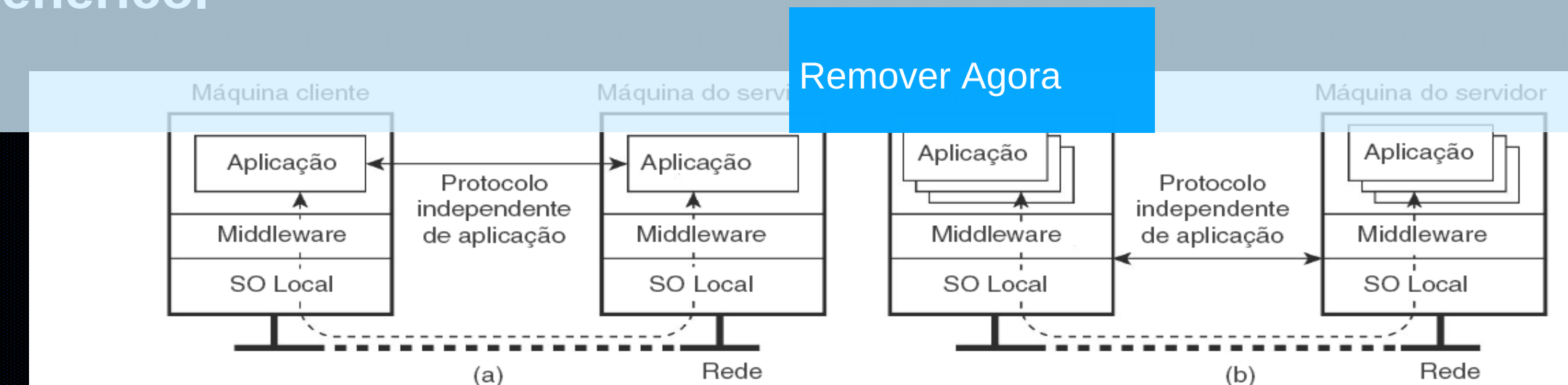
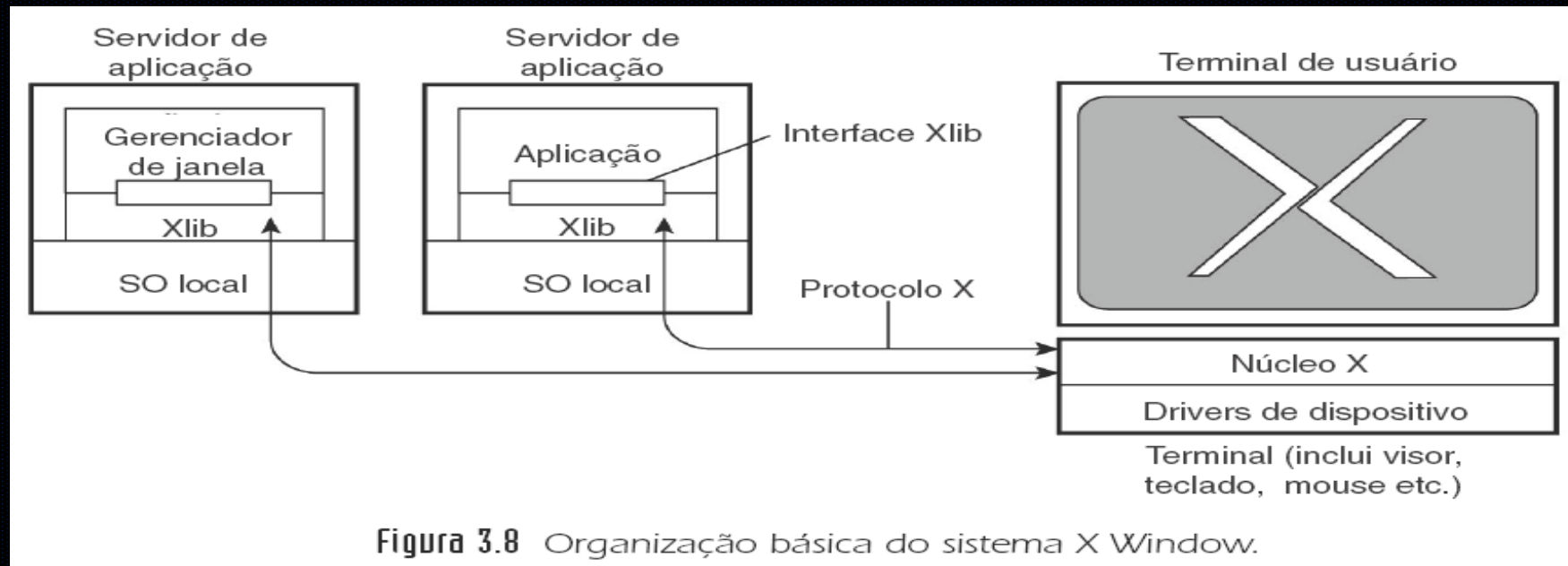


Figura 3.7 (a) Aplicação em rede com seu próprio protocolo. (b) Solução geral para permitir acesso a aplicações remotas.

Clientes: modos de interação com o servidor

- X Window é uma implementação de **interface de usuário em rede**.
- Implementação de um cliente minimizado (*thin client*).



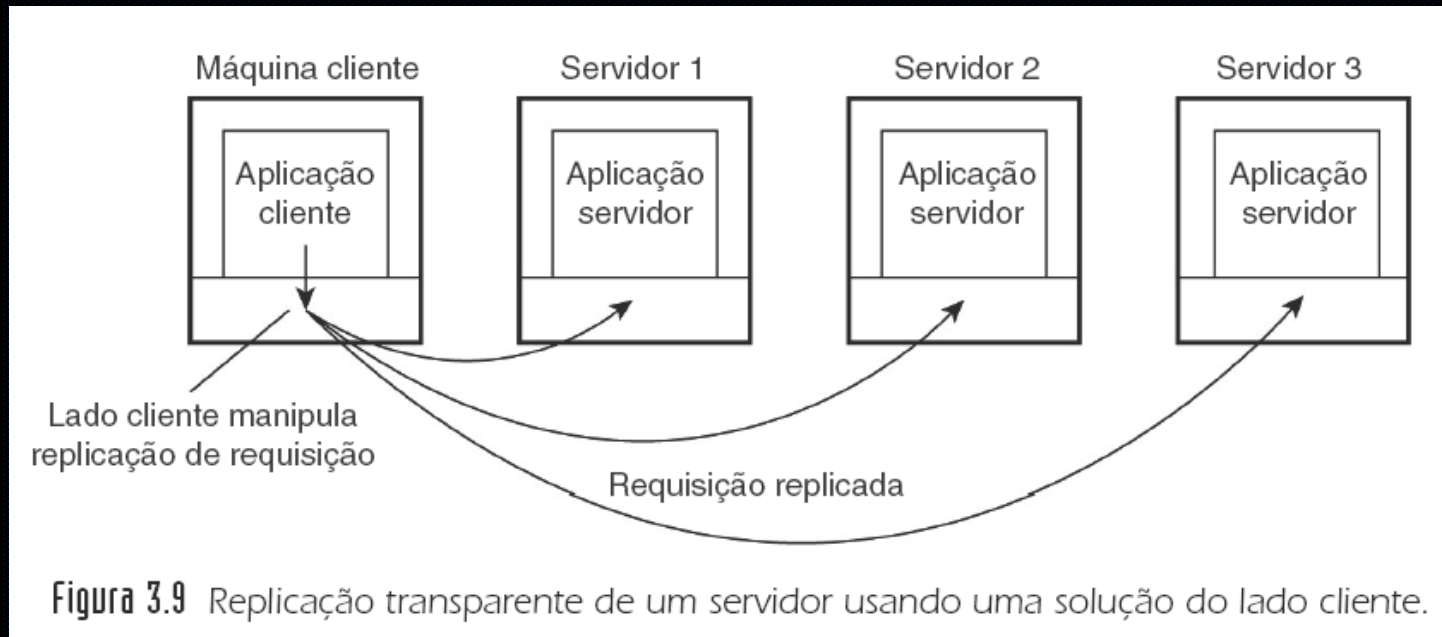
Clientes - Transparência

Implementação no lado do cliente:

- **Transparência de Migração:** O middleware no cliente oculta do usuário a corrente localização do servidor, caso este migre para outro sítio.
- **Transparência a falha:** O middleware pode tentar a conexão com um servidor repetidas vezes ou tentar um outro servidor após n tentativas.
 - Uso de cache: falha na conexão com o servidor.

Clientes - Transparência

- **Transparência de replicação:** várias réplicas de requisição e uma resposta é enviada a aplicação.
- Oculta isso do cliente
- Desempenho?



Servidores – Considerações

- Como tratar as requisições?
- Como os clientes contatam um servidor?
- Como interromper a comunicação com o servidor?
- Servidor deve guardar estado da comunicação dos clientes?

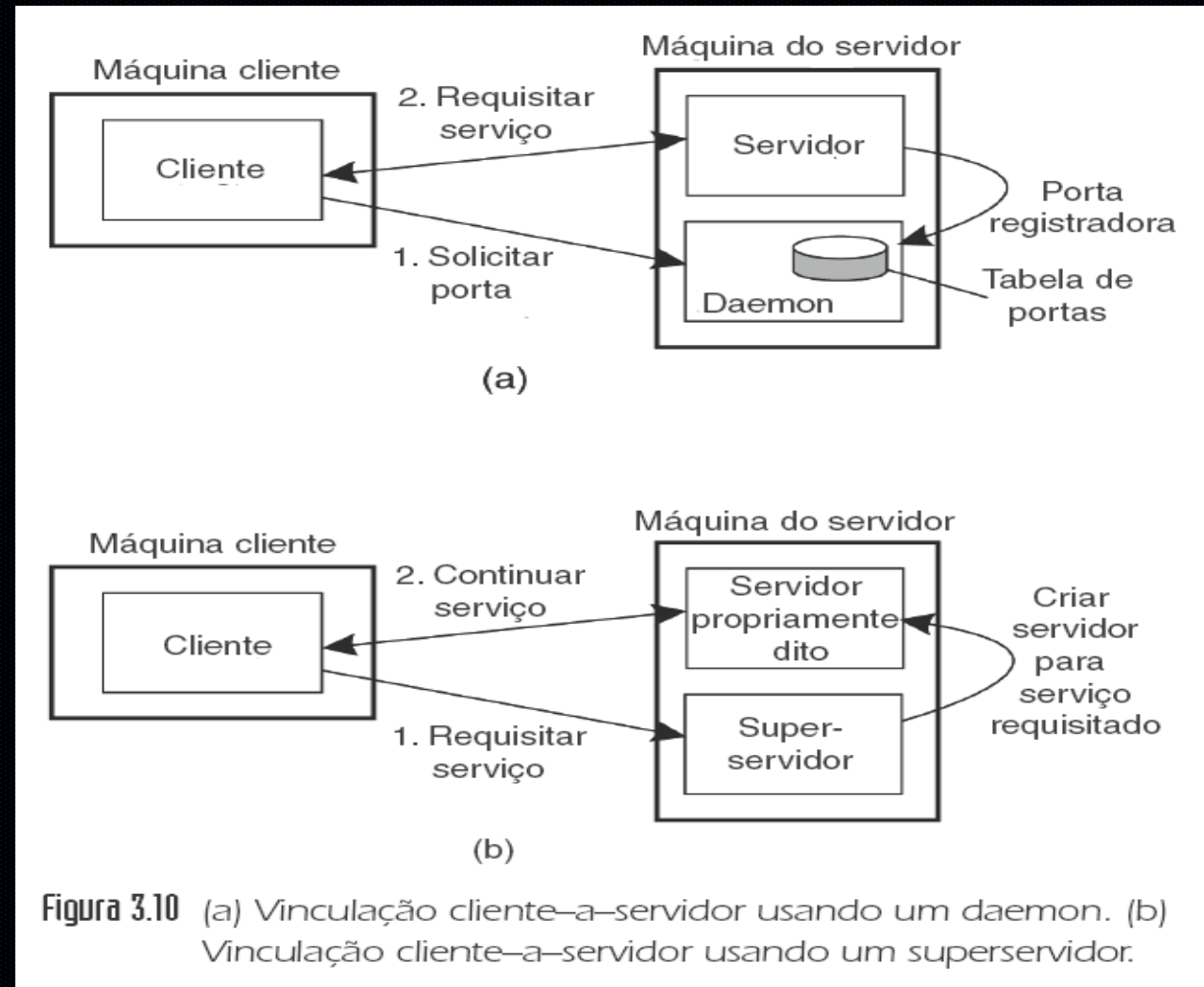
Servidores: Tratamento de Requisições

- **Servidor Iterativo**
 - Próprio servidor manipula a requisição
 - Responde ao cliente quando necessário.
- **Servidor Concorrente**
 - Não manipula por si próprio a requisição
 - Ex.: Servidor multithread
 - Processos ou threads respondem ao cliente.

Servidores: Como cliente contata?

(a) Um daemon informa o # da porta.

(b) Um superservidor bifurca a chamada de uma porta para o servidor correto.



Servidores: Interrompendo a Comunicação

- **Abordagem mais simples:** usuário sai abruptamente da aplicação cliente → servidor encerrará a conexão antiga.
- **Abordagem mais completa:** dados “urgentes” (fora da banda)
 - Pacotes TCP possuem o campo URG no header
 - Servidor ao receber um pacote com o campo URG setado é interrompido para tratá-lo.
 - Exemplo: Segmentos Telnet.

Servidores: Manutenção de Estado

Três implementações:

- Sem estado
- Estado flexível
- Com estado

Servidores sem estado

- Não mantém informações sobre os estados dos clientes. Por exemplo, com o protocolo HTTP.
- Servidor Web: Após processar uma requisição, esquece o cliente (e.g. não guarda os acessos dele).
- Mesmo sem estado, servidor Web guarda informações de clientes.
- Uma alternativa para armazenar estados é com os *cookies*.

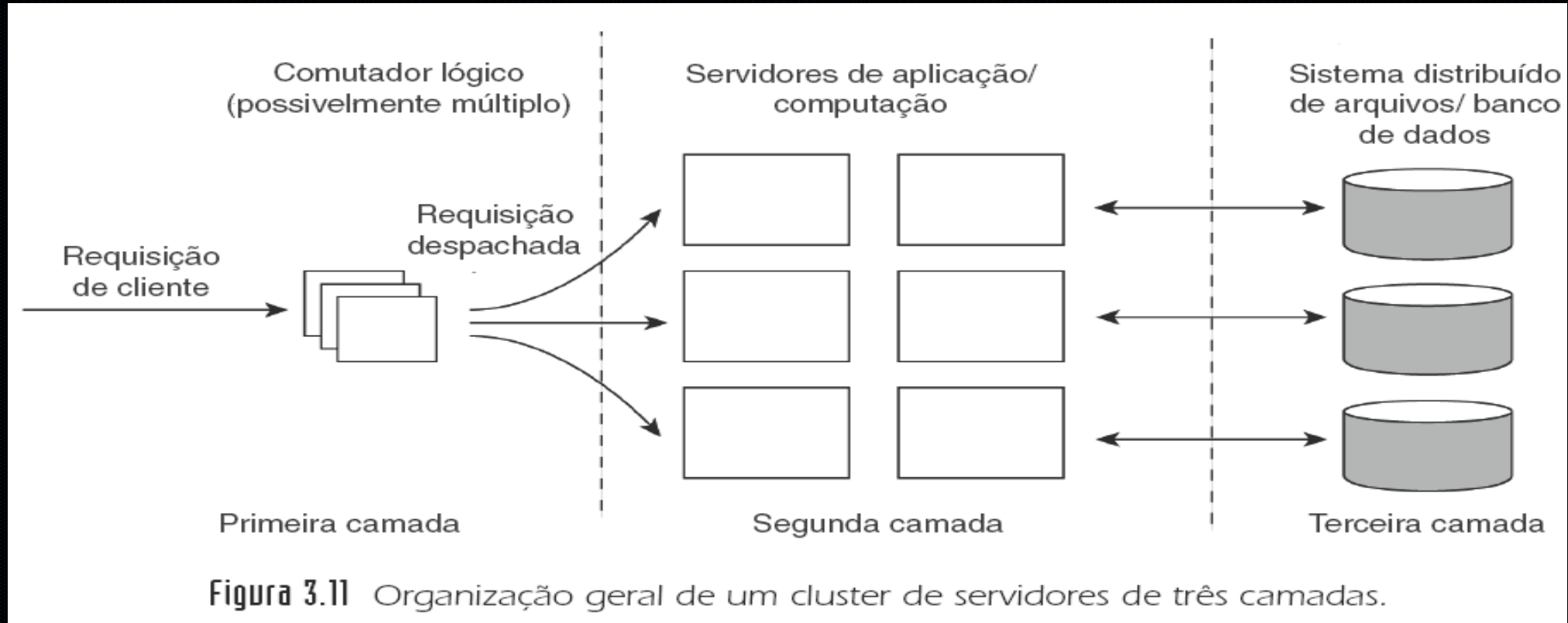
Servidores com estado flexível

- Servidor promete manter estado em nome do cliente por tempo limitado.
- Após o tempo limitado, o servidor descarta estas informações.
- Ex.: Servidor promete manter um cliente informado sobre as atualizações.

Servidores com estado

- Mantém informações persistentes sobre seus clientes.
- Ex.: Servidor de arquivos que permite ao cliente manter cópia “extra” local de um arquivo.
- Servidor mantém uma tabela com entradas (*cliente, arquivo*)
- Monitora as permissões sobre um arquivo, versão atual, etc.
- Desafio: tolerância a falhas para manter todas as informações.

Clusters de servidores



Migração de Código

Migrar códigos de uma máquina para outra

- Principal razão: **aumento de desempenho**
- Envio de processos para máquinas menos sobrecarregadas
- **Evitar grande quantidade de mensagens trocadas** entre aplicações cliente-servidor:
 - Ex.1: Operações de banco de dados que envolvem uma grande quantidade de troca de dados.
 - Ex.2: Formulários enviados do servidor para cliente.

Modelos para Migração de Código

- **Mobilidade fraca:** Transfere apenas o **segmento de código** e alguns dados de inicialização
- Requer somente que a máquina-alvo possa executar o código (portabilidade).
- **Mobilidade forte: Segmento de execução** (e.g. pilha, PC) também pode ser transferido
- O processo em execução pode ser parado, movido para uma outra máquina e retomar a execução no ponto original.

Migração em relação ao ponto de início da migração

- **Iniciada pelo remetente:** A migração é iniciada na máquina em que o código está em execução
 - Ex.: Enviar programa de busca pela Internet a um servidor de banco de dados para buscas.
- **Iniciada pelo destinatário:** A iniciativa da migração de código é tomada pela máquina-alvo.

Concluindo

Foram abordadas nesta aula:

- **Implementações dos processos distribuídos**

Estes slides estão baseados na bibliografia:

Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V. (2ª edição)

Na próxima aula, veremos:

- **Comunicação Distribuída**

AULA Nº 06

SISTEMAS DISTRIBUÍDOS

Implementações dos Processos Distribuídos

Os slides são baseados no livro de Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V.